

An MPC-Based Framework for Dynamic Trajectory Re-Planning in Uncertain Environments

Maolin Lei^{1,2}, Liang Lu¹, Arturo Laurenzi¹, Luca Rossini^{1,2}, Edoardo Romiti¹,
Jörn Malzahn¹, and Nikos G. Tsagarakis¹

Abstract—Online motion re-planning is an important feature for introducing robots into unstructured environments where the close presence of humans at any time can challenge the operation of the robot from the human safety perspective. This work introduces a novel re-planning framework for robotic manipulators operating in dynamic environments where the interactions with humans may occur either in an anticipated or unexpected manner. The contribution of the proposed framework lies in the fact that it allows to account for the uncertainty of human pose and challenges associated with human motion estimation during occlusion phases of the human with respect to the perception system on the robot. To this aim the proposed framework is comprised of an uncertainty estimation component and a model predictive control (MPC) component, the combination of which enables to efficiently and dynamically track a task-space trajectory by the robot while limiting the probability of potential collisions with a moving human obstacle entering the workspace of the robot. Simulations and experimental trials on a robotic platform show the effectiveness of the proposed framework in re-planning the trajectory of the robotic arm under the presence of a human detected by the perception system installed on the robot.

I. INTRODUCTION

In human-robot collaboration scenarios, online trajectory re-planning for collaborative robotic arms has attracted much attention due to the need to ensure the safety of humans interacting with the robot while the robot is executing the task. Such interventions of the human inside the robot workspace can naturally occur as part of the collaborative task executed by the robot and the human together, but they can also occur in an unexpected manner when humans and robots work independently while they need to coexist in the same environment. In both scenarios when humans are moving inside the workspace of the robot, the uncertainty in the human presence detection and pose state, due to the perception system noise and occlusions, can compromise the operation of the robot resulting to collisions and eventually violation of the safety requirements.

Therefore, this uncertainty in the human presence detection and pose state needs to be considered in the trajectory re-planning when a human is detected, and the online re-planning needs to be fast enough to incorporate the latest perception information available from the environment and the potential presence of the human within it.

¹ Humanoids and Human Centered Mechatronics Research Line, Istituto Italiano Di Tecnologia (IIT), 16163 Genoa, Italy. e-mail: name.surname@iit.it

² Department of Informatics, Bioengineering, Robotics, and Systems Engineering (DIBRIS), University of Genova, Genova, Italy.

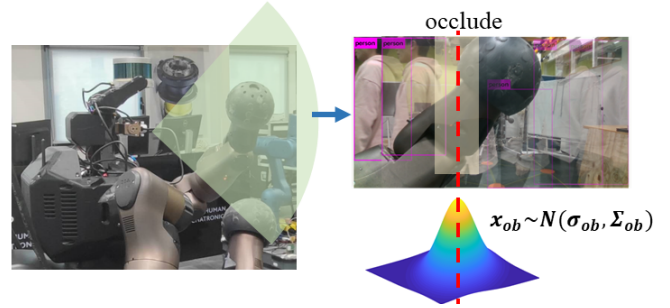


Fig. 1: Representation of the pose uncertainty of a moving obstacle.

Motivated by this, in this paper we introduce a dynamic uncertainty-aware trajectory re-planning framework, which can improve the robot re-planning performance in avoiding potential collisions with humans who move inside the robot workspace. We consider scenarios, where the robot shall keep executing its motion task while at the same time, a human can enter the robot's working area with a probability and the human presence and pose estimated by the perception system, contains also a level of uncertainty due to the perception system noise and short times occlusions that may occur. Compared with the detection of static objects, dynamic object perception statistics are characterized by several uncertainties, such as:

- (i) detection uncertainty: the robot may occlude the sensor's view in a short time at some special configuration when the robot tracks the desired task;
- (ii) future state uncertainty: the information available at the present time cannot fully represent the future state of the moving object.

These uncertainties can eventually result in a collision between the robot and the human, giving rise to a *collision probability*, which is in general non negligible.

To address these uncertainties in the re-planning problem, we propose an approach based on *Model Predictive Control (MPC)*. The rationality of the global trajectory in task space is guaranteed by algorithms in [1], [2] and [3], considering that the robot can execute the task in an ideal manner. However, safely tracking a defined task-space trajectory requires the robot to have accurate joint control, while also keeping the collision probability under a specified threshold. In this work, we introduce an MPC-based controller to adjust the robot's joint velocity in real-time according to the actual task space's position, velocity, and human's pose states. We choose to model the perception uncertainty of a moving

human by means of an *Extended Kalman Filter (EKF)*, which allows to quantify the probability associated with a future human state, therefore enabling the MPC to explicitly bound the collision probability below a user-specified threshold. The schematic diagram of the overall proposed on-line re-planning framework is depicted in Figure 2.

The main contributions of this paper are the following features:

- The integration of the Gaussian filtering approach with MPC framework to estimate the obstacle's¹ probabilistic state even in the case of temporary occlusions, exploiting its future state prediction to improve safety.
- The use of a collision-probability-limited minimum-jerk trajectory planning in task space with implicit Inverse Kinematics (IK) solution.
- The real-time capability of the online re-planning framework is demonstrated in simulation and experiment.

The rest of the paper is structured as follows. Section II discusses the related work, Section III introduces the dynamic object estimation and prediction, Section IV presents the details of the trajectory generation and MPC formulation while Section V presents the simulation and experimental trials performed. Finally Section VI addresses the conclusions of this work.

II. RELATED WORK

The methods for online trajectory planning under collision constraints can be generally classified into two categories: *Artificial Potential Field (APF)* based methods, and optimization based methods.

A. APF-based method

APF methods have been widely used to deal with dynamic environments, which was first proposed in [4] and tested on a mobile robot. To use it on robotic arms, the works in [5], [6] have applied a virtual attractive force to control the robot's end-effector, while a repulsive force acts on the arm links in order to avoid collisions. The APF method was also extended to human-robot collaborative scenarios [7]. These methods can re-plan the robot's trajectory in the presence of dynamic obstacles, yet they are known to suffer from local minima issues, which may cause the robot to remain stuck at a configuration that is far from the intended goal. To mitigate this problem, the previous work [8] projected the repulsive velocity in the direction of the current end-effector velocity; however, this work can not ensure that the robot will complete the final task. Meanwhile, the work in [9] proposed a planner that, by combining the local APF with global elastic planning, can reduce the probability of falling into a local minimum point. Even though these works [8], [9] manage to mitigate convergence to local minima, the robot still shows jittery behavior around some points.

¹We use 'obstacle' to replace 'human' in the remainder of this paper

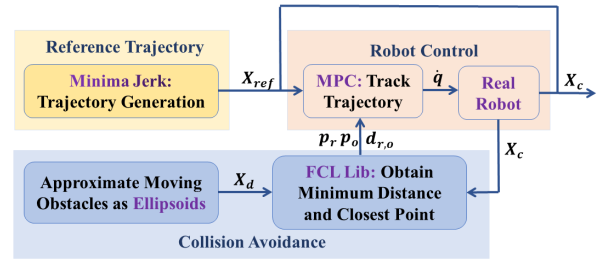


Fig. 2: Overview schematic of the proposed re-planning framework.

B. Optimization-based method

Optimization-based trajectory planning methods can solve the desired joint motion online while satisfying the constraints acting on the system and avoiding the jittery problem caused by using APF methods [10]. To take full advantage of the optimization-based methods, most works choose to plan the robotic arm's trajectory in configuration space (C-space).

The planners proposed by [11], [7], [12], [13], [14] rely on *Nonlinear Model Predictive Control (NMPC)* to generate the safe trajectories in real-time in a dynamic environment. However, solving the *Non-Linear Programs (NLP)* associated with NMPC can place a high computational burden on the system [15]. To improve computational efficiency works such as [10] resort to a linearization of the nonlinear constraints, whereas the work in [16] transforms the nonlinear constraint into a soft constraint to be penalized inside the cost function. Furthermore, to handle Cartesian space tasks, the robot needs to solve the IK at several desired way-points, which greatly consumes the computational sources. Apart from [13], the known references did not predict the obstacle's motion, which may lead to the collision in the future time instants. The work described in [13] did not account for temporary occlusions of the obstacle, which renders predictive collision avoidance difficult. A solution to both aspects represents a contribution to our re-planning framework.

III. DYNAMIC OBJECT ESTIMATION AND PREDICTION

As stated in the previous section (see also Fig. 1), dynamic object pose estimation is affected by two kinds of uncertainty, which stem from obstacle detection failures (caused by e.g. the robot occlude the sensor's field of view, or due to a failure in the detection algorithm), and the object's future motion's un-prediction. In view of this, the detection system can be divided into three main parts, as will be introduced in this section.

A. Dynamic Obstacle Detection

In our work, we use an onboard RGBD camera sensor to detect the obstacle. By training the *YOLO* architecture [17] on the the *Pascal VOC* data sets ², high frame rate can be achieved if a GPU can be used. When the obstacles fall into the view of the camera, our trained detector can capture the obstacle and provide as output a bounding box that encloses the obstacle, plus the pose information expressed w.r.t. the

²<http://host.robots.ox.ac.uk/pascal/VOC/>

camera frame $\mathcal{C}$. To improve the detection accuracy, we use the *K-mean++* clustering algorithm [18] to obtain a smoother obstacle pose estimation that is affected less by the noise. In this work, as the human obstacle is assumed to walk on flat terrain; we take the assumption of fixed altitude to design the detection part. Considering the size of the obstacle body, instead of taking account of one point, we take n points from the center line of the bounding box to describe the position of the obstacle in the image. The position of the obstacle is then mapped from the image to the 2D scan, while the orientation of the obstacle is obtained from the previously observed and the current position. To estimate the current pose of the obstacle in the camera frame, we use polar coordinates to express it as:

$$\begin{aligned}\theta_i^o &= \arctan\left(\frac{u_{x_i} - c_x}{f_x}\right) \\ x_i^o &= d_i^o \cos(\theta_i^o) \\ y_i^o &= d_i^o \sin(\theta_i^o) \\ \psi_i^o &= \arctan\left(\frac{y_{i+1}^o - y_i^o}{x_{i+1}^o - x_i^o}\right)\end{aligned}\quad (1)$$

where u_x represents the object x position in image coordinates (pixels), c_x represents the image center (pixels), and f_x is the camera focal length; furthermore, d_i^o represents the 2D Euclidean distance from the camera to the detected obstacle and $\mathbf{X}_i^o = [x_i^o, y_i^o, \psi_i^o]$ represents the current pose of the obstacle in the camera frame; The obstacle pose only has three components because first, the human does not have pitch or roll, second under the assumption of this work, we consider the altitude fixed. $(\cdot)_{i+1}$ represents the state of $(\cdot)$ at time $i + 1$. $\mathbf{X}_i^o$ is then transferred to the world frame to obtain the pose of the obstacle.

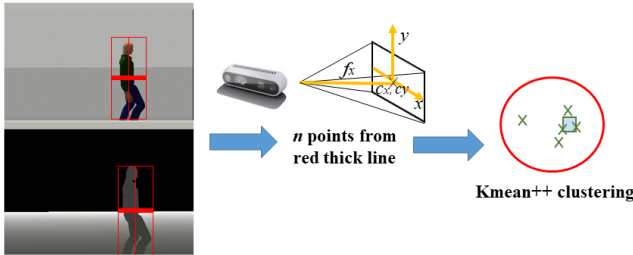


Fig. 3: The obstacle pose estimation pipeline.

B. Obstacle prediction and approximation

Based on the real-time feedback of the object pose, its predicted motion can finally be computed. While most methods are performing this by solving a nonlinear system [19], [20], and therefore need significant computation time, we decide to use a more efficient Extended Kalman Filter (EKF)-based method that considers linearized prediction and correction models as in [21]. However, the obstacle's pose cannot be estimated in case of miss-detection failures by the perception pipeline, and therefore the prediction state's output of obstacle is discontinuous when using the method in [21]. We therefore improve this EKF-based approach for

predicting the obstacle's future state based on the object pose history.

To this end, assume the position $\mathbf{x}_{ob} \sim \mathcal{N}(\boldsymbol{\mu}_p, \boldsymbol{\Sigma}_p)$ and velocity $\mathbf{v}_{ob} \sim \mathcal{N}(\boldsymbol{\mu}_v, \boldsymbol{\Sigma}_v)$ of the obstacle's center w.r.t. the world frame to be normally distributed where $\boldsymbol{\mu}_p = [x_{ob,x}, x_{ob,y}, x_{ob,z}]^T \in \mathbb{R}^3$ and $\boldsymbol{\mu}_v = [v_{ob,x}, v_{ob,y}, v_{ob,z}]^T \in \mathbb{R}^3$ is the expectation of position and velocity, respectively; , therefore, $\boldsymbol{\Sigma}_x$ and $\boldsymbol{\Sigma}_v$ represents the uncertainty covariance matrix of position and velocity, respectively. Furthermore, the obstacle's geometry is assumed to be described by a constant-size bounding box with size given by $\mathbf{s}_o = [l, w, h]^T \sim \mathcal{N}(\boldsymbol{\mu}_o, \boldsymbol{\Sigma}_o)$, as shown in Fig. 3. Similar to the method in [21], over a short control time, the obstacle is assumed to move at a constant velocity while modifying the original bounding box as the ellipsoid. Under this assumption, the obstacle's state can be predicted in the linear form and the resulting ellipsoid can be formed:

$$\begin{aligned}\boldsymbol{\mu}_p^{k+1} &= \boldsymbol{\mu}_p^k + \boldsymbol{\mu}_v^k dt, \quad \boldsymbol{\mu}_v^{k+1} = \boldsymbol{\mu}_v^k \\ \boldsymbol{\sigma}_x^{k+1} &= \boldsymbol{\Sigma}_x^k + \boldsymbol{\Sigma}_v^k dt^2, \quad \boldsymbol{\Sigma}_v^{k+1} = \boldsymbol{\Sigma}_v^k \\ \mathbf{s}_o^{k+1} &= \mathbf{s}_o^k, \quad \mathbf{a}_o^{k+1} = \frac{\sqrt{3}}{2} \mathbf{s}_o^{k+1}\end{aligned}\quad (2)$$

where apex k represents discrete time, and $\mathbf{a}_o = [a, b, c]^T$ represents the size of the bounding ellipsoid. Then, using the EKF to fuse the predictions with the observed information to predict future positions can greatly reduce the variance of future information and be described as

$$\begin{aligned}\hat{\mathbf{x}}_{ob}^{k+1} &= \mathbf{F}_k \hat{\mathbf{x}}_{ob}^k + \mathbf{B}_k \mathbf{u}_{ob}^k \\ \hat{\mathbf{y}}_{ob}^k &= \mathbf{H}_k \hat{\mathbf{x}}_{ob}^k\end{aligned}\quad (3)$$

where $\mathbf{F}_k, \mathbf{B}_k$ represents process matrices and observation matrices of the EKF, respectively. $\mathbf{H}_k$ is the matrix of the observation model. In case of detection failures, only the historical state of the obstacle can be used to predict the obstacle's motion, as shown in Eq. 2, but with higher covariance compared with the EKF's prediction information.

The probability for future state threshold is set as $\delta = 0.03$ (thus confidence level 0.97), which corresponds to the 3σ confidence ellipsoid [22], named probability ellipsoid. Under this assumption, the ellipsoid box will be enlarged to include the state within the probabilities at the confidence level. Compared to the ellipsoid constructed by the EKF-based method, the size of the ellipsoid is larger if only using the Eq. 2 to predict the future state of the obstacle.

C. Distance Estimation and Gradient Calculation

To rapidly measure the distance between the robot and the probability ellipsoid, we use cylinder collision models to describe the links of the robot, whereas the *Flexible Collision Library (FCL)* [23] library is used to compute collision state, and closest points between any pair of collision shapes. Given the closest points $\mathbf{p}_r \in \mathbb{R}^3$ and $\mathbf{p}_o \in \mathbb{R}^3$, the distance between the corresponding collision shapes is given by

$$g(\mathbf{q}) = d_{ro}^2 = \|\mathbf{P}_{r,o}\|_2^2 = \|\mathbf{p}_r - \mathbf{p}_o\|_2^2 \quad (4)$$

where $\mathbf{q}$ represents the configuration of the robot, d_{ro} represents the Euclidean distance between the robot and obstacle,

$\|\cdot\|_2$ represents the 2-norm, $\mathbf{P}_{r,o} = \mathbf{p}_r - \mathbf{p}_o \in \mathbb{R}^3$ represents the vector between the nearest point, with $\mathbf{p}_r \in \mathbb{R}^3$ and $\mathbf{p}_o \in \mathbb{R}^3$ represent the point on the root and the object, respectively. Since the vector $\mathbf{p}_o$ is constant, the gradient relative to time t of the vector between the nearest point can be calculated as

$$\frac{d\mathbf{P}_{r,o}}{dt} = \frac{d\mathbf{p}_r}{dq} \frac{dq}{dt} = \mathbf{J}_n \dot{\mathbf{q}}_n \quad (5)$$

where $\mathbf{J}_n = \frac{d\mathbf{p}_r}{dq} \in \mathbb{R}^{3 \times n_c}$ is the Jacobian matrix of the nearest point on the robot expressed in the world frame $\mathbf{W}$, $\dot{\mathbf{q}}_n \in \mathbb{R}^{n_c}$ represents the joint velocity before the nearest point, and n represents a link identifier.

IV. TRAJECTORY GENERATION AND MPC FORMULATION

This section introduces the methodology used for trajectory generation in task space, which happens before our MPC stage, and the formulation of the MPC itself. MPC is considered to track the robot's reference trajectory in task space while keeping the collision probability under a user-defined threshold (collision-probability-limitation). At the same time, in our method, the robot can use the MPC to track the desired trajectory in the task space directly without needing to solve the IK at waypoints, which significantly improves computational efficiency.

A. Trajectory Generation in Task Space

The global trajectory generation in task space was decomposed into two parts, i.e. position and orientation, each of which integrates the physical constraints in terms of task space position, velocity, and acceleration limits.

1) *Position trajectory generation*: The position part is constructed from a minimum-jerk argument since jerk influences the smoothness of a trajectory [24]. Inspired by the work of [1], [2], the position trajectory can be divided into multiple segments, each of which can be described as a polynomial of the following form:

$$\mathbf{P}(t) = \sum_{i=1}^n p_i B_{i,t}(t) \quad (6)$$

where Eq.(6) represents a single trajectory segment between two points in the flat output variables x, y, z , with respect to time t , n represents the number of terms of the polynomial. $B_{i,t}(t)$ is the time function for each term of the polynomial; and p_i represents the weight of each polynomial.

The minimum-jerk cost of one trajectory segment can be written as

$$\mathbf{J}_{ta}(T_j) = \int_{T_j}^{T_{j+1}} \mathbf{P}^{(3)}(t)^2 dt \quad (7)$$

where T_j and T_{j+1} represent the initial and final time, respectively of the j^{th} segment trajectory; $(\cdot)^{(3)}$ represents the 3-rd order derivative of $(\cdot)$. The optimization problem

for solving the whole trajectory can be formulated:

$$\begin{aligned} \min_{\mathbf{p}_i} \quad & \sum_{j=1}^m \mathbf{J}(T_j) \\ \text{s.t.} \quad & \mathbf{v}_{min} \leq \mathbf{v} \leq \mathbf{v}_{max} \\ & \mathbf{a}_{min} \leq \mathbf{a} \leq \mathbf{a}_{max} \\ & \mathbf{d}_{t,j} = \mathbf{d}_{0,j+1} \\ & \mathbf{P}(t) \in \mathbb{S} \\ & i = 0, 1, 2, \dots, n \\ & j = 0, 1, 2, \dots, m \end{aligned} \quad (8)$$

where $\mathbf{J}(T_i)$ is the cost function penalizing the squares of the derivatives of $\mathbf{P}(t)$ relative to time t ; $\mathbf{v}_{min}$, $\mathbf{a}_{min}$, $\mathbf{v}_{max}$, $\mathbf{a}_{max}$ represent constraints of velocity and acceleration, respectively; $\mathbf{d}_{t,j}$ represents the derivatives at the end of the j^{th} segment while $\mathbf{d}_{0,j+1}$ represents the derivatives at the beginning of the next segment; $\mathbb{S}$ represents the reachable workspace of the robot, which was obtained by using the method in [25]. The trapezoidal time allocation is applied to calculate time t_j at j^{th} segment, which was written as

$$t_j = T_{j+1} - T_j = \frac{v_{max}^2 + l_j a_{max}}{a_{max} v_{max}} \quad (9)$$

where l_j is the distance between the initial and endpoint of the trajectory segment j .

2) *Orientation trajectory generation*: Inspired by the work of [3], the orientation trajectory is generated according to

$$\mathbf{R}(\zeta) = \mathbf{R}_{in} \exp(\zeta \log(\mathbf{R}_{in}^T \mathbf{R}_d)) \quad \zeta \in [0, 1] \quad (10)$$

where $\zeta = \zeta(t)$ is a function of time t ; $\mathbf{R}_{in} \in SO(3)$ and $\mathbf{R}_d \in SO(3)$ represent the initial orientation and desired orientation, respectively; $\mathbf{R}_{in}^T \mathbf{R}_d$ is an expression of the orientation error between the initial and desired orientation, whereas the logarithm $\log(\cdot)$ maps an element of $SO(3)$ into its Lie algebra [26], [27]; exponentiation mapping $\exp(\cdot)$ is $\log(\cdot)$'s inverse function, so that $\mathbf{R}(\zeta) \in SO(3)$ represents the desired orientation at the specified time.

B. Kinematics-based Modeling

For our MPC, we select the internal model state $\mathbf{X}$ to be given by

$$\mathbf{X} := [\mathbf{p} \quad \mathbf{o} \quad \dot{\mathbf{p}} \quad \boldsymbol{\omega}]^T \in \mathbb{R}^{13} \quad (11)$$

where, $\mathbf{p} \in \mathbb{R}^3$, $\dot{\mathbf{p}} \in \mathbb{R}^3$, represent the position and linear velocity of the end-effector of the robot expressed in the world frame $\mathbf{W}$, respectively; the quaternion $\mathbf{o} = [\eta, \boldsymbol{\epsilon}] \in \mathbb{R}^4$ represents the relative orientation with $\|\mathbf{o}\| = 1$, $\eta = o_w$, $\boldsymbol{\epsilon} = [o_x, o_y, o_z]^T$ expressed in the world frame $\mathbf{W}$; $\boldsymbol{\omega} \in \mathbb{R}^3$ represents the relative angular velocity expressed in the world frame $\mathbf{W}$.

According to the kinematic model of the robot, we can use the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{6 \times n_j}$, expressed w.r.t. the world frame $\mathbf{W}$, to reflect the joints velocities $\dot{\mathbf{q}} \in \mathbb{R}^{n_j}$ into an end-effector velocity $\mathbf{v}_e \in \mathbb{R}^6$, which can therefore be written as:

$$\mathbf{J} \dot{\mathbf{q}} = \mathbf{v}_e \quad (12)$$

where $\mathbf{J} = [\mathbf{J}_v \ \mathbf{J}_\omega]^T$; $\mathbf{J}_v \in \mathbb{R}^{3 \times n_j}$ represents the linear velocity part of the Jacobian, $\mathbf{J}_\omega \in \mathbb{R}^{3 \times n_j}$ represents the angular velocity part of the Jacobian, and n_j is the number of degrees of freedom for the robot at hand.

The acceleration of the end-effector $\dot{\mathbf{v}}_e \in \mathbb{R}^6$ can be computed by using the Jacobian matrix $\mathbf{J}$ and the differential Jacobian matrix $\dot{\mathbf{J}} \in \mathbb{R}^{6 \times n_j}$, as follows:

$$\dot{\mathbf{J}}\dot{\mathbf{q}} + \mathbf{J}\ddot{\mathbf{q}} = \dot{\mathbf{v}}_e \quad (13)$$

Eq. (12) and (13) can be rewritten as,

$$\begin{bmatrix} \mathbf{v}_{e,k} \\ \dot{\mathbf{v}}_{e,k} \end{bmatrix} = \begin{bmatrix} \mathbf{J}_k \dot{\mathbf{q}}_k \\ \dot{\mathbf{J}}_k \dot{\mathbf{q}}_k + \mathbf{J}_k \ddot{\mathbf{q}}_k \end{bmatrix} \quad (14)$$

where k represents the node index of the MPC's prediction horizon. If constant joint acceleration is assumed between nodes, the joint acceleration $\ddot{\mathbf{q}}_k$ vector can be written as

$$\ddot{\mathbf{q}}_k \approx \frac{\dot{\mathbf{q}}_k - \dot{\mathbf{q}}_{k-1}}{dt} \in \mathbb{R}^{n_j} \quad (15)$$

Combining Eq. (14) and (15), (14) can be rewritten as

$$\begin{bmatrix} \mathbf{v}_{e,k} \\ \dot{\mathbf{v}}_{e,k} \end{bmatrix} = \begin{bmatrix} \mathbf{J}_k \dot{\mathbf{q}}_k \\ \dot{\mathbf{J}}_k \dot{\mathbf{q}}_k + \mathbf{J}_k (\dot{\mathbf{q}}_k - \dot{\mathbf{q}}_{k-1})/dt \end{bmatrix}, \quad (16)$$

leading to the continuous time state-space equation that follows:

$$\dot{\mathbf{X}}_k = \begin{bmatrix} \dot{\mathbf{p}}_k \\ \dot{\mathbf{o}}_k \\ \dot{\mathbf{p}}_k \\ \dot{\mathbf{\omega}}_k \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{p}}_k \\ \mathbf{o}_k[0, \omega_k/2]^T \\ \dot{\mathbf{J}}_{v,k} \dot{\mathbf{q}}_k + \mathbf{J}_{v,k} (\dot{\mathbf{q}}_k - \dot{\mathbf{q}}_{k-1})/dt \\ \dot{\mathbf{J}}_{\omega,k} \dot{\mathbf{q}}_k + \mathbf{J}_{\omega,k} (\dot{\mathbf{q}}_k - \dot{\mathbf{q}}_{k-1})/dt \end{bmatrix}. \quad (17)$$

We define the joint velocity vector $\dot{\mathbf{q}}_k$ as the input of such model. The discretized state-space equation according to a *Forward Euler* integration rule with step size dt is then given by the following expression:

$$\mathbf{X}_{k+1} = \begin{bmatrix} \mathbf{p}_{k+1} \\ \mathbf{o}_{k+1} \\ \dot{\mathbf{p}}_{k+1} \\ \dot{\mathbf{\omega}}_{k+1} \end{bmatrix} = \begin{bmatrix} \mathbf{p}_k + \dot{\mathbf{p}}_k dt \\ \mathbf{o}_k + \mathbf{o}_k[0, \omega_k/2]^T dt \\ \mathbf{v}_k + \dot{\mathbf{J}}_k \dot{\mathbf{q}}_k dt + \mathbf{J}_k (\dot{\mathbf{q}}_k - \dot{\mathbf{q}}_{k-1}) \\ \mathbf{\omega}_k + \dot{\mathbf{J}}_k \dot{\mathbf{q}}_k dt + \mathbf{J}_k (\dot{\mathbf{q}}_k - \dot{\mathbf{q}}_{k-1}) \end{bmatrix} \quad (18)$$

C. Constraints

1) *Constraint For Joint Position and Acceleration:* Similar to the Eq.(15), the velocity of the joint can be approximated as:

$$\dot{\mathbf{q}}_k \approx \frac{\mathbf{q}_k - \mathbf{q}_{k-1}}{dt}; \quad (19)$$

Eq. (19) can be then rewritten as

$$\mathbf{q}_l \leq \mathbf{q}_k \approx \dot{\mathbf{q}}_k dt + \mathbf{q}_{k-1} \leq \mathbf{q}_u \in \mathbb{R}^{n_j}, \quad (20)$$

where $\mathbf{q}_l$ and $\mathbf{q}_u$ represent the upper and lower position limits, respectively. A constraint on the acceleration can be written as

$$\ddot{\mathbf{q}}_l \leq \ddot{\mathbf{q}}_k \approx \frac{\dot{\mathbf{q}}_k - \dot{\mathbf{q}}_{k-1}}{dt} \leq \ddot{\mathbf{q}}_u \in \mathbb{R}^{n_j} \quad (21)$$

where $\ddot{\mathbf{q}}_l$ and $\ddot{\mathbf{q}}_u$ represents the upper and lower acceleration limits, respectively. Eq.(20) and Eq.(21) can be written as a linear constraint function with respect to $\dot{\mathbf{q}}$:

$$\mathbf{u}_l \leq \mathbf{A}_{c,k} \dot{\mathbf{q}}_k + \mathbf{b}_{c,k} \leq \mathbf{u}_u \quad (22)$$

with:

$$\mathbf{A}_{c,k} = \begin{bmatrix} \mathbf{E} dt \\ \mathbf{E}/dt \end{bmatrix}, \mathbf{b}_{c,k} = \begin{bmatrix} \dot{\mathbf{q}}_{k-1} \\ -\dot{\mathbf{q}}_{k-1}/dt \end{bmatrix}, \quad (23)$$

$$\mathbf{u}_l = \begin{bmatrix} \mathbf{q}_l \\ \ddot{\mathbf{q}}_l \end{bmatrix}, \mathbf{u}_u = \begin{bmatrix} \mathbf{q}_u \\ \ddot{\mathbf{q}}_u \end{bmatrix}$$

where $\mathbf{E} \in \mathbb{R}^{3 \times 3}$ represents the identity matrix. Therefore, the linear constraint for acceleration and position can be formed.

2) *Collision Avoidance Constraints:* To bound the collision probability under a given threshold, each link of the robot should be enforced to keep a safe distance from the probability ellipsoid. According to Eq. (4), a collision constraint can be formed as

$$g(\mathbf{q}) = d_{ro}^2 = \|\mathbf{p}_{r,k+t} - \mathbf{p}_{o,k+t}\|_2^2 \geq d_{min}^2 \quad (24)$$

where d_{min} represents the threshold of the distance between the robot and the obstacle; $\mathbf{p}_{r,k+t}$, $\mathbf{p}_{o,k+t}$ represent the predicted position at time t of the robot and obstacle, respectively. The predicted position of the obstacle $\mathbf{p}_{o,k+t}$ can be obtained in Eq. (3). According to the gradient of the nearest vector, the nearest distance is able to be predicted at gradient direction(Eq. 5) in the short time dt as

$$g(\dot{\mathbf{q}}) = \|\mathbf{J}_n \dot{\mathbf{q}}_n dt + \mathbf{p}_{r,k} - \mathbf{p}_{o,k+t}\|_2^2 \geq d_{min}^2 \quad (25)$$

which means that the robot could keep a safety distance with the obstacle at the future step $k+t$.

To write Eq.25 as linear, the Eq.(25) can be approximated by the well-known Taylor expansions around the current joint velocity $\dot{\mathbf{q}}_0$ and be approximated as,

$$g(\dot{\mathbf{q}}) \approx g(\dot{\mathbf{q}}_0) + \nabla g(\dot{\mathbf{q}}_0)(\dot{\mathbf{q}} - \dot{\mathbf{q}}_0) = g(\dot{\mathbf{q}}_0) + 2g(\dot{\mathbf{q}}_0)\mathbf{J}_n \dot{\mathbf{q}}_0(\dot{\mathbf{q}} - \dot{\mathbf{q}}_0) \geq d_{min}^2 \quad (26)$$

where the $\nabla g(\dot{\mathbf{q}})$ represents the gradient of $g(\dot{\mathbf{q}})$. In this way, the constraint with collision avoidance was written as the linear constraint while constraints for collision avoidance in the prediction horizon can be constructed.

D. MPC Formulation

The basic optimization formulation of MPC can be formed as:

$$\min_{\mathbf{u}} \sum_{k=1}^N (\|\mathbf{X}(k+1) - \mathbf{X}_{ref}(k+1)\|_Q + \|\mathbf{u}(k)\|_P)$$

$$\text{s.t. : } \mathbf{X}(k+1) = \mathbf{A}_k \mathbf{X}(k) + \mathbf{B}_k \mathbf{u}(k)$$

$$\mathbf{q}_{min} \leq \mathbf{q} \leq \mathbf{q}_{max}$$

$$\dot{\mathbf{q}}_{min} \leq \dot{\mathbf{q}} \leq \dot{\mathbf{q}}_{max}$$

$$\ddot{\mathbf{q}}_{min} \leq \ddot{\mathbf{q}} \leq \ddot{\mathbf{q}}_{max}$$

$$\forall \ m \in M$$

$$g(\dot{\mathbf{q}}_0) + \sum_{k=1}^m 2g(\dot{\mathbf{q}}_0)\mathbf{J}_n \dot{\mathbf{q}}_0(\dot{\mathbf{q}}_k - \dot{\mathbf{q}}_0) \geq d_{min}^2 \quad (27)$$

where there are two parts in the cost function dedicated to tracking the desired trajectory with weight Q in task space and minimizing the joint energy with Q simultaneously, $\mathbf{X}_{ref}$ represents the reference trajectory of the robot. The error of the position, velocity and angular velocity are, $e_{pk} = \mathbf{p}_k - \mathbf{p}_{dk}$, $e_{\dot{p}_k} = \dot{\mathbf{p}}_k - \dot{\mathbf{p}}_{dk}$, and $e_{\omega_k} = \omega_k - \omega_{dk}$, respectively. $e_{o_k} = \eta_k \epsilon_d - \eta_d \epsilon_k - [\epsilon_d]^\times \epsilon_k$ represents the orientation error. To enable the optimization to find a feasible solution, we consider the relaxation of collision constraints, which means that the collision avoidance constraint is only considered in the next M -th ($M \leq N$) control horizon rather than the whole MPC's predicted horizon. The Eq.(27) was finally written as the QP standard formulation:

$$\begin{aligned} \min_{\mathbf{u}} \quad & \frac{1}{2} \mathbf{u}^T \mathbf{W} \mathbf{u} + \mathbf{c}^T \mathbf{u} \\ \text{s.t.} \quad & \mathbf{u}_{leq} \leq \mathbf{A}_{ieq} \mathbf{u} \leq \mathbf{u}_{ueq} \\ & \mathbf{A}_{eq} \mathbf{u} + \mathbf{b}_{eq} = 0 \end{aligned} \quad (28)$$

Among several off-the-shelf solvers for QP problems, we chose qpOASES [28] in this paper.

V. SIMULATIONS AND EXPERIMENTS

Simulation and experimental trials were performed to assess the effectiveness of the proposed motion trajectory re-planning framework. The simulation experiments use the Robot Operating System (ROS) running on top of Ubuntu 20.04 on a laptop with a CPU of Intel Core i5-10300H at 2.2 GHz with 8 cores. The Gazebo simulator is used to realize the environment model. The robot physical model used in the simulation is a CENTAURO robot model while the human physical model used the actuator model from Gazebo Simulation³. The simulation trajectory of the human model is predefined in the world file of the Gazebo. The proposed framework has been implemented also on the robot CENTAURO [29], which is powered by the XBotCore real-time middleware [30], and is equipped with an Intel Realsense D453i camera installed on its head. The algorithmic implantation of the proposed method was realized in C++, and run on a Linux machine. Unless otherwise stated, the parameters used in the following simulations and experiments are $\mathbf{Q} = \text{diag}[3.5, 3.5, 3.5, 0.05, 0.05, 0.05, 3.5, 3.5, 3.5, 0.05, 0.05, 0.05]$, $\mathbf{P} = \text{diag}[0.002, 0.002, 0.002, 0.002, 0.002]$, $\mathbf{F} = [0.002 \times \mathbf{E}_{3 \times 3}; \mathbf{0}_{3 \times 3}]$, $\mathbf{B} = [0.002 \mathbf{E}_{3 \times 3}; \mathbf{0}_{3 \times 3}]$, $\mathbf{H} = [\mathbf{E}_{3 \times 3}, \mathbf{0}_{3 \times 3}]$, $dt = 0.02s$, $N = 5$, $d_{min} = 0.02m$, respectively in the following tests. The value of the parameter in EKF is not changing during the whole period.

A. Uncertainty estimation comparison study and results

This subsection introduces the simulation in estimating the uncertainty of the obstacle and compares the real with the estimated position of the moving obstacle, no matter whether the sensor was occluded.

In this trial, the obstacle moves on the line repeatedly and the perception camera was used to detect the pose of the obstacle. We compare our method with the approach reported

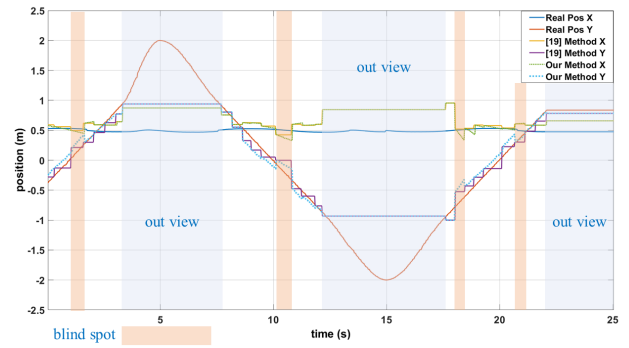


Fig. 4: Comparison results for estimating the uncertainty obstacle. The red line and blue line represent the x , and y position of the ground truth. The orange line and purple line represent the x , and y position produced by the method in [21]. The green dashed line and the blue dashed line represents the x , and y of the predicted position generated by the proposed method.

in [21]. Irrespective of the method, the sensor will no longer detect the obstacle when the obstacle is outside the sensor's field of view. The results of the comparison between the real obstacle's position and the estimated position are illustrated in Fig. 4, which shows that the position prediction provides a smoother output compared with that of [21], even when the obstacle is temporarily occluded.

B. Trajectory Tracking and re-planning

The method in [21] does not take into account the occlusion problem. Consequently, the temporal resolution of the resulting obstacle's trajectory is low. On the contrary, our method predicts the position of the obstacle across even longer horizons corresponding to the covariance. This can eventually enable more dynamic and accurate obstacle avoidance using our method.

1) *Unobstructed Trajectory Tracking*: To show the effectiveness of the framework in tracking a given trajectory, we first use the real robot to track the desired trajectory without disturbing it with the introduction of the human obstacle. We recorded the trajectory data including the error between the desired position and orientation of the robot, named $e = \mathbf{X} - \mathbf{X}_{ref}$, in the task space with respect to the world frame, as illustrated in Fig.5. In this obstacle-free environment, the position error between the desired and

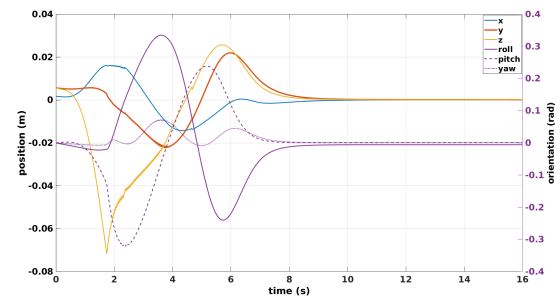
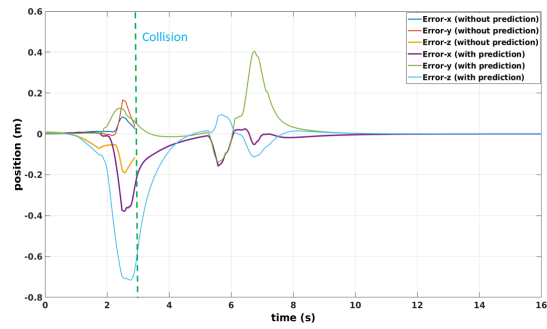
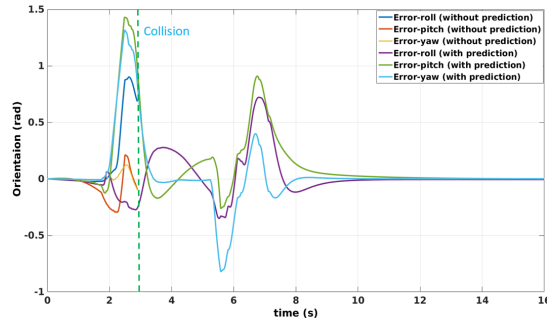


Fig. 5: The position and orientation data for the robot used in no obstacle environment. The left y -axis shows the error of the position tracking while the right y -axis with the purple color represents the error of the orientation tracking.

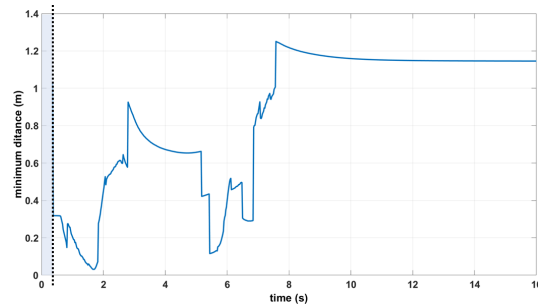
³<https://github.com/captjulian/HumanWalkingTrajSimulation.git>



(a) The data of position tracking in the simulation. The dark blue, red, and orange lines show the x , y , and z position tracking error without the prediction of the obstacle while the light blue, green, and purple lines show the position tracking error with the prediction of the obstacle.



(b) The data of orientation tracking in the simulation. The dark blue, red, and orange lines show the roll, pitch, and yaw orientation tracking error without the prediction of the obstacle while the light blue, green, and purple lines show the position tracking error with the prediction of the obstacle.



(c) Minimum distance between the robot and the obstacle. The shadow before the dotted line in the figure represents the obstacle have not appeared in the camera's view.

Fig. 6: Results from the simulation trials performed to evaluate the trajectory tracking and re-planning of the proposed method.

realized trajectory on the x , y , z axis remains below 0.003 m, while the orientation error in pitch, yaw, and roll is below 0.02 rad.

2) *Trajectory Tracking with Collision Probability:* To illustrate the effect of the uncertainty estimation proposed in Section III, two comparative simulations were conducted: *i*) The robot is using MPC to track the desired trajectory without considering the uncertainty. *ii*) The robot is using MPC to track the desired trajectory while considering the uncertainty of the obstacle location. During the robot motion, we introduce a moving obstacle into the workspace. Fig.6a and 6b displays the error between actual and desired pose

of the robot in task space. When the obstacle comes close to the robot, the robot performs re-planning of the trajectory and avoids collision with the introduced obstacle. Once the obstacle moves out of the robot's path, the robot approaches and continues to track the initially desired trajectory, as shown in Fig.6a and 6b. As can be seen, the error between the desired and actual poses rapidly decreases once the obstacle moves away. Furthermore, the minimum distance between the robot and the obstacle in this process is presented in Fig. 6c. The consideration of the obstacle uncertainty estimation in the proposed framework enables more safely dealing with the presence of the obstacle compared to the methods that do not take obstacle uncertainty estimation into account. More details about this comparison can be seen in the accompanying video.

3) *Trajectory Tracking while Introducing an Obstacle:* To further demonstrate the efficacy of the proposed framework, we use the robot to track the trajectory in task space while a human moves forward and enters the workspace of the robot. The robot successfully performs the re-planning of the task trajectory avoiding the collision with the human and continues to track the desired trajectory once the human goes away from the robot workspace. Snapshots from this experiment are illustrated in Fig. 7 while the execution of this experiment can be also observed in the accompanying video.

VI. CONCLUSIONS

This paper proposes a probabilistic real-time framework for re-planning the trajectories of a robot arm in the presence of obstacles interfering with the robot workspace. It is an optimization-based method, which predicts the motion of obstacles, extrapolating across temporary occlusions. A QP solver is used to generate the trajectory with minimum-jerk, considering the physical constraints of the robot, while a Gaussian filter (EKF in this case) predicts the obstacle's future motion. The main idea behind the uncertainty estimation is that the prediction model for the obstacle is linear, which improves the computation speed. Finally, the formulated MPC problem yields the required joint velocities to realize the trajectory in the task space without collisions with the obstacle. Simulations and experiments verify the efficacy of the proposed framework with which online re-planning in uncertain environments can be achieved.

Our future work with extends the current framework in the re-planning of trajectories for whole-body loco-manipulation tasks.

ACKNOWLEDGMENTS

This work is supported from the Istituto Nazionale per l'Assicurazione contro gli Infortuni sul Lavoro (INAIL) and European Union's Horizon 2020 programme (grant Agreement No. 101016007 CONCERT).

REFERENCES

- [1] C. Richter, A. Bry, and N. Roy, "Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments," in *Robotics research*, pp. 649–666, Springer, 2016.



Fig. 7: Snapshots of the experiment executed using the arm of the Centauro robot.

- [2] L. Lu, A. Carrio, C. Sampedro, and P. Campoy, "A robust and fast collision-avoidance approach for micro aerial vehicles using a depth sensor," *Remote Sensing*, vol. 13, no. 9, 2021.
- [3] G. Li, D. Song, S. Xu, L. Sun, and J. Liu, "Kinematic-free orientation control for a deformable manipulator based on the geodesic in rotation group so (3)," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 2432–2438, 2018.
- [4] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Autonomous robot vehicles*, pp. 396–404, Springer, 1986.
- [5] S. M. Khansari-Zadeh and A. Billard, "A dynamical system approach to realtime obstacle avoidance," *Autonomous Robots*, vol. 32, no. 4, pp. 433–454, 2012.
- [6] F. Flacco and A. De Luca, "Safe physical human-robot collaboration," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 2072–2072, IEEE, 2013.
- [7] A. Oleinikov, S. Kusdavitov, A. Shintemirov, and M. Rubagotti, "Safety-aware nonlinear model predictive control for physical human-robot interaction," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5665–5672, 2021.
- [8] M. Lei, T. Wang, C. Yao, H. Liu, Z. Wang, and Y. Deng, "Real-time kinematics-based self-collision avoidance algorithm for dual-arm robots," *Applied Sciences*, vol. 10, no. 17, p. 5893, 2020.
- [9] S. J. Moon, J. Kim, H. Yim, Y. Kim, and H. R. Choi, "Real-time obstacle avoidance using dual-type proximity sensor for safe human-robot interaction," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 8021–8028, 2021.
- [10] A. Pupa, M. Arrfou, G. Andreoni, and C. Secchi, "A safety-aware kinodynamic architecture for human-robot collaboration," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4465–4471, 2021.
- [11] M. Krämer, C. Rösmann, F. Hoffmann, and T. Bertram, "Model predictive control of a collaborative manipulator considering dynamic obstacles," *Optimal Control Applications and Methods*, vol. 41, no. 4, pp. 1211–1232, 2020.
- [12] S. Trimble, W. Naeem, S. McLoone, and P. Sotasakis, "Context-aware robotic arm using fast embedded model predictive control," in *2020 31st Irish Signals and Systems Conference (ISSC)*, pp. 1–6, IEEE, 2020.
- [13] A. Kanazawa, J. Kinugawa, and K. Kosuge, "Adaptive motion planning for a collaborative robot based on prediction uncertainty to enhance human safety and work efficiency," *IEEE Transactions on Robotics*, vol. 35, no. 4, pp. 817–832, 2019.
- [14] M. Lei, M. Selvaggio, T. Wang, F. Ruggiero, C. Zhou, C. Yao, and Y. Zheng, "Dual-arm object transportation via model predictive control and external disturbance estimation,"
- [15] Y. Hu, H. Su, J. Fu, H. R. Karimi, G. Ferrigno, E. De Momi, and A. Knoll, "Nonlinear model predictive control for mobile medical robot using neural optimization," *IEEE Transactions on Industrial Electronics*, vol. 68, no. 12, pp. 12636–12645, 2020.
- [16] M. Krämer, R. J. Velasco-Guillen, P. Beckerle, and T. Bertram, "Comparing online robot joint space trajectory optimization for task space applications," in *2021 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, pp. 223–230, IEEE, 2021.
- [17] T. L. Dang, G. T. Nguyen, and T. Cao, "Object tracking using improved deep sort yolov3 architecture," *ICIC Express Letters*, vol. 14, no. 10, pp. 961–969, 2020.
- [18] A. Likas, N. Vlassis, and J. J. Verbeek, "The global k-means clustering algorithm," *Pattern recognition*, vol. 36, no. 2, pp. 451–461, 2003.
- [19] K. Driggs-Campbell, R. Dong, and R. Bajcsy, "Robust, informative human-in-the-loop predictions via empirical reachable sets," *IEEE Transactions on Intelligent Vehicles*, vol. 3, no. 3, pp. 300–309, 2018.
- [20] D. Fridovich-Keil, A. Bajcsy, J. F. Fisac, S. L. Herbert, S. Wang, A. D. Dragan, and C. J. Tomlin, "Confidence-aware motion prediction for real-time collision avoidance," *The International Journal of Robotics Research*, vol. 39, no. 2-3, pp. 250–265, 2020.
- [21] J. Lin, H. Zhu, and J. Alonso-Mora, "Robust vision-based obstacle avoidance for micro aerial vehicles in dynamic environments," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2682–2688, IEEE, 2020.
- [22] M. Bensimhoun, "N-dimensional cumulative function, and other useful facts about gaussians and normal densities," *Jerusalem, Israel, Tech. Rep.*, pp. 1–8, 2009.
- [23] J. Pan, S. Chitta, and D. Manocha, "Fcl: A general purpose library for collision and proximity queries," in *2012 IEEE International Conference on Robotics and Automation*, pp. 3859–3866, IEEE, 2012.
- [24] C. Dai, S. Lefebvre, K.-M. Yu, J. M. Geraedts, and C. C. Wang, "Planning jerk-optimized trajectory with discrete time constraints for redundant robots," *IEEE Transactions on Automation Science and Engineering*, vol. 17, no. 4, pp. 1711–1724, 2020.
- [25] G. Li, F. Caponetto, E. Del Bianco, V. Katsageorgiou, I. Sarakoglou, and N. G. Tsagarakis, "A workspace limit approach for teleoperation based on signed distance function," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5589–5596, 2021.
- [26] M. Fourmy, T. Flayols, N. Mansard, and J. Solà, "Contact forces pre-integration for the whole body estimation of legged robots," in *IEEE International Conference on Robotics and Automation-ICRA 2021*, 2021.
- [27] F. Bullo and A. D. Lewis, *Geometric control of mechanical systems: modeling, analysis, and design for simple mechanical control systems*, vol. 49, Springer, 2019.
- [28] H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl, "qpOases: A parametric active-set algorithm for quadratic programming," *Mathematical Programming Computation*, vol. 6, no. 4, pp. 327–363, 2014.
- [29] N. Kashiri, L. Baccelliere, L. Muratore, A. Laurenzi, Z. Ren, E. M. Hoffman, M. Kamedula, G. F. Rigano, J. Malzahn, S. Cordasco, et al., "Centauro: A hybrid locomotion and high power resilient manipulation platform," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1595–1602, 2019.
- [30] L. Muratore, A. Laurenzi, E. M. Hoffman, A. Rocchi, D. G. Caldwell, and N. G. Tsagarakis, "Xbotcore: A real-time cross-robot software platform," in *2017 First IEEE International Conference on Robotic Computing (IRC)*, pp. 77–80, IEEE, 2017.